

模块 0：核心机制 — AI 编程代理的“发动机”

模块概述

本模块是整个课程的起点，也是理解后续所有高级策略的基础。我们将从两个视角切入：

- 工程视角：**一个 AI 编程代理（如 Claude Code）在接到任务后，究竟做了什么？我们将逐帧拆解一次完整的任务执行过程，从系统提示词加载到最终验证，不遗漏任何细节。
- 算法视角：**将代理的行为抽象为序列决策（Sequential Decision-Making）框架，用 MDP/POMDP 的语言精确描述“编程”这件事，揭示当前系统的本质结构与局限。

时长：2-2.5 小时

结构分配：

环节	占比	内容
机制讲解	30%	核心循环、MDP 建模、策略空间
论文/算法解读	20%	ReAct、Chain-of-Thought
工程演示	20%	Claude Code 实际 trace 分析
动手实操	30%	标注练习、对比实验

前置知识：学员应具备 LLM 基础使用经验，了解 Transformer 架构的基本原理，有一定编程经验（Python/TypeScript 优先）。

0.1 从一个真实任务说起

问题设定

假设我们有一个 Python Web 应用，用户报告了一个 bug：

“注册接口在用户名包含特殊字符（如 @、#）时返回 500 错误，而不是友好的 400 验证提示。”

我们把这个任务交给 Claude Code，看看它究竟做了什么。

完整 Trace 逐帧分析

以下是一个高度真实的 Claude Code 执行 trace。每一步我们都会标注其在核心循环中所处的阶段（将在 0.2 节正式定义）。

第一帧：系统初始化（System Prompt Loading）

```
[System] Loading system prompt...  
[System] Reading CLAUDE.md from project root...  
[System] CLAUDE.md loaded: 45 lines of project-specific instructions  
[System] Environment: Python 3.11, Django 4.2, pytest  
[System] Working directory: /app/backend
```

发生了什么： 在用户输入任何内容之前，Claude Code 先加载了多层上下文：

- 系统提示词（System Prompt）：** 定义了代理的角色、可用工具列表、行为约束（如安全性要求、权限模型）。这是代理的“操作系统”。
- 项目指令（CLAUDE.md）：** 项目级别的约定——技术栈、代码风格、测试要求、特殊注意事项。这是代理的“项目手册”。
- 环境信息：** 工作目录、平台、Shell 类型等元数据。

关键洞察： 这一步决定了代理的“初始信念状态”（initial belief state）。高质量的 CLAUDE.md 能显著提升代理效率——这不是玄学，而是信息论的直接推论：更好的先验 → 更少的探索 → 更快收敛到正确方案。

第二帧：接收用户指令（Intent Parsing）

User: 注册接口在用户名包含特殊字符时返回 500 而不是 400 验证错误，请修复。

Assistant (thinking): 用户报告了一个输入验证 bug。我需要：

- 找到注册接口的代码
- 理解当前的输入验证逻辑
- 找到导致 500 错误的原因
- 添加适当的验证
- 确保修复有测试覆盖

阶段：Intent（意图理解）

代理将自然语言任务转化为内部的行动计划草案。注意这里的“thinking”不是简单的文本生成——它是一个将模糊需求映射到具体工程步骤的推理过程。

第三帧：定位代码（Grounding — 感知当前状态）

```
Tool Call: Glob
  pattern: "**/*register*"
```

```
Result:
  app/views/register.py
  app/tests/test_register.py
  app/forms/register_form.py
```

```
Tool Call: Grep
  pattern: "def register"
  glob: "*.py"
```

```
Result:
  app/views/register.py:45: def register_user(request):
  app/views/register.py:78: def register_confirm(request):
```

阶段：Grounding（感知）

代理没有“记住”整个代码库——它需要主动探索。这里的策略选择至关重要：

- 它先用 `Glob` 做文件级别的模式匹配（快速、低成本）
- 再用 `Grep` 做内容搜索（精确定位）
- 这种“先粗后细”的搜索策略（coarse-to-fine search）是一个经典的信息获取模式

工程细节：为什么不直接用 `Bash` 执行 `grep -r`？专用的 `Grep` 工具底层同样是命令行工具（`ripgrep`），但关键区别在于**接口设计**：LLM 不需要自己拼命令字符串（容易出错——flag 写错、转义遗漏、路径错误），只需填写结构化参数（`pattern`、`glob`），框架负责处理执行细节。这是 ACI（Agent-Computer Interface）设计的核心思想——同样的底层能力，一层对 LLM 友好的接口就能显著降低出错率。

第四帧：阅读代码（Grounding — 深入感知）

```
Tool Call: Read
  file_path: "/app/backend/app/views/register.py"
```

```
Result:
```

```

1  from django.http import JsonResponse
2  from django.views.decorators.http import require_POST
3  from django.contrib.auth.models import User
4  import json
5
...
45 @require_POST
46 def register_user(request):
47     try:
48         data = json.loads(request.body)
49         username = data.get('username', '')
50         password = data.get('password', '')
51
52         # Create user directly without validation
53         user = User.objects.create_user(
54             username=username,
55             password=password
56         )
57         return JsonResponse({'id': user.id, 'username':
user.username}, status=201)
58     except Exception as e:
59         return JsonResponse({'error': str(e)}, status=500)

```

阶段：Grounding（深度感知）

现在代理看到了问题的全貌：

1. **没有输入验证**： `username` 直接传给 `create_user`，没有任何格式检查
2. **异常处理过于粗暴**：所有异常都返回 500，包括本应是客户端错误（4xx）的情况
3. **Django 的 `create_user` 在遇到非法字符时会抛出 `ValueError`**，这被 `except Exception` 捕获后返回了 500

这是一个典型的“缺失验证层”问题。

第五帧：阅读测试文件（Grounding — 补充上下文）

```

Tool Call: Read
file_path: "/app/backend/app/tests/test_register.py"

Result:
1  from django.test import TestCase, Client
2
3  class RegisterTestCase(TestCase):
4      def setUp(self):
5          self.client = Client()
6
7      def test_register_success(self):

```

```

8         response = self.client.post('/api/register/',
9             data='{"username": "testuser", "password":
"securepass123"}',
10            content_type='application/json')
11         self.assertEqual(response.status_code, 201)
12
13     def test_register_duplicate(self):
14         self.client.post('/api/register/',
15             data='{"username": "testuser", "password": "pass123"}',
16             content_type='application/json')
17         response = self.client.post('/api/register/',
18             data='{"username": "testuser", "password": "pass456"}',
19             content_type='application/json')
20         self.assertEqual(response.status_code, 500) # Bug: should
be 400/409

```

阶段：Grounding（持续感知）

代理注意到现有测试也反映了这个 bug——`test_register_duplicate` 居然断言 500 状态码，说明这个问题可能长期存在且被“接受”了。

关键观察：代理在修复代码之前先看了测试，这体现了一个重要的工程实践——理解现有测试覆盖情况，避免引入回归（regression，即软件质量的倒退：修了 A 却把 B 搞坏了，俗话说“按下葫芦浮起瓢”）。

但要警惕：代理把测试当作“标准答案”来验证自己的修复。如果测试本身就是错的呢？上面的例子中 `test_register_duplicate` 断言了 500 状态码——这本身就是一个错误的测试（应该是 409）。如果代理不加判断地“让所有测试通过”，它会按照错误的预期去写代码。这揭示了 Feedback 环节的一个根本性风险：**当评价函数本身不可靠时，代理会“认真地做错事”**。好的代理应该能识别出“测试本身有 bug”这种情况——这也是为什么 Feedback 策略不能只依赖测试，还需要模型自评和人工审查作为补充。

第六帧：制定修复方案（Planning）

Assistant (thinking):

分析完成。修复方案：

1. 添加用户名验证函数（只允许字母、数字、下划线、连字符）
2. 在 `register_user` 中调用验证，不通过则返回 400
3. 区分不同类型的异常：
 - 验证失败 → 400
 - 用户名重复 → 409
 - 其他错误 → 500
4. 更新测试以覆盖新的验证逻辑

阶段：Planning（规划）

注意规划的粒度：代理没有止步于“添加验证”这种模糊描述，而是给出了具体的技术决策（正则表达式、HTTP 状态码选择、异常分层策略）。这种具体性直接影响后续 action 的质量。

第七帧：执行修复（Action）

```
Tool Call: Edit
file_path: "/app/backend/app/views/register.py"
old_string: "from django.http import JsonResponse
from django.views.decorators.http import require_POST
from django.contrib.auth.models import User
import json"
new_string: "from django.http import JsonResponse
from django.views.decorators.http import require_POST
from django.contrib.auth.models import User
from django.db import IntegrityError
import json
import re

def validate_username(username):
    if not username or len(username) < 3 or len(username) > 30:
        return False, 'Username must be between 3 and 30 characters'
    if not re.match(r'^[a-zA-Z0-9_-]+$ ', username):
        return False, 'Username can only contain letters, numbers,
underscores, and hyphens'
    return True, ''"
```

```
Tool Call: Edit
file_path: "/app/backend/app/views/register.py"
old_string: "@require_POST
def register_user(request):
    try:
        data = json.loads(request.body)
        username = data.get('username', '')
        password = data.get('password', '')

        # Create user directly without validation
        user = User.objects.create_user(
            username=username,
            password=password
        )
        return JsonResponse({'id': user.id, 'username': user.username},
status=201)
    except Exception as e:
        return JsonResponse({'error': str(e)}, status=500)"
```

```

new_string: "@require_POST
def register_user(request):
    try:
        data = json.loads(request.body)
        username = data.get('username', '')
        password = data.get('password', '')

        is_valid, error_msg = validate_username(username)
        if not is_valid:
            return JsonResponse({'error': error_msg}, status=400)

        if not password or len(password) < 8:
            return JsonResponse({'error': 'Password must be at least 8
characters'}, status=400)

        user = User.objects.create_user(
            username=username,
            password=password
        )
        return JsonResponse({'id': user.id, 'username': user.username},
status=201)
    except IntegrityError:
        return JsonResponse({'error': 'Username already exists'},
status=409)
    except json.JSONDecodeError:
        return JsonResponse({'error': 'Invalid JSON in request body'},
status=400)
    except Exception as e:
        return JsonResponse({'error': 'Internal server error'},
status=500)"

```

阶段：Action（执行）

代理使用 `Edit` 工具而非 `Bash` 中的 `sed` ——这是工程规范的体现。`Edit` 工具要求精确匹配 `old_string`，这提供了一种隐式的“乐观锁”机制：如果文件在代理读取后被修改了，编辑会失败而不是静默覆盖。

修复的质量值得注意：- 添加了输入验证函数（可复用）- 使用了正则表达式（精确的字符集控制）- 区分了 `IntegrityError`（409）和 `JSONDecodeError`（400）- 保留了兜底的 `Exception` 处理，但返回通用错误信息而非泄露内部细节（安全意识）

第八帧：更新测试（Action — 续）

```

Tool Call: Edit
file_path: "/app/backend/app/tests/test_register.py"
old_string: "class RegisterTestCase(TestCase):
    def setUp(self):

```

```

        self.client = Client()

    def test_register_success(self):
        ..."
new_string: "class RegisterTestCase(TestCase):
    def setUp(self):
        self.client = Client()

    def test_register_success(self):
        response = self.client.post('/api/register/',
            data='{"username": "testuser", "password": "securepass123"}',
            content_type='application/json')
        self.assertEqual(response.status_code, 201)

    def test_register_special_chars_rejected(self):
        response = self.client.post('/api/register/',
            data='{"username": "user@name", "password": "securepass123"}',
            content_type='application/json')
        self.assertEqual(response.status_code, 400)
        self.assertIn('error', response.json())

    def test_register_short_username_rejected(self):
        response = self.client.post('/api/register/',
            data='{"username": "ab", "password": "securepass123"}',
            content_type='application/json')
        self.assertEqual(response.status_code, 400)

    def test_register_short_password_rejected(self):
        response = self.client.post('/api/register/',
            data='{"username": "validuser", "password": "short"}',
            content_type='application/json')
        self.assertEqual(response.status_code, 400)

    def test_register_duplicate(self):
        self.client.post('/api/register/',
            data='{"username": "testuser", "password": "securepass123"}',
            content_type='application/json')
        response = self.client.post('/api/register/',
            data='{"username": "testuser", "password": "anotherpass123"}',
            content_type='application/json')
        self.assertEqual(response.status_code, 409)

    def test_register_invalid_json(self):
        response = self.client.post('/api/register/',
            data='not valid json',
            content_type='application/json')
        self.assertEqual(response.status_code, 400)"

```

阶段：Action（继续执行）

代理不仅修复了代码，还更新了测试套件： - 修正了原来错误的 500 断言 - 新增了针对特殊字符、短用户名、短密码、无效 JSON 的测试用例 - 每个测试用例对应一个验证规则——这是良好的测试设计

第九帧：运行测试（Grounding — 观察结果）

```
Tool Call: Bash
  command: "cd /app/backend && python -m pytest app/tests/test_register.py -v"

Result:
  app/tests/test_register.py::RegisterTestCase::test_register_success
  PASSED
  app/tests/
  test_register.py::RegisterTestCase::test_register_special_chars_rejected
  PASSED
  app/tests/
  test_register.py::RegisterTestCase::test_register_short_username_rejected
  PASSED
  app/tests/
  test_register.py::RegisterTestCase::test_register_short_password_rejected
  PASSED
  app/tests/test_register.py::RegisterTestCase::test_register_duplicate
  PASSED
  app/tests/test_register.py::RegisterTestCase::test_register_invalid_json
  PASSED

  6 passed in 0.42s
```

阶段：Grounding（结果观察）+ Feedback（反馈）

测试全部通过。这一步既是“感知”（观察测试结果），也是“反馈”（测试结果验证了修复的正确性）。

注意代理使用了 `Bash` 工具来运行测试——这是 `Bash` 的正确使用场景（执行需要 shell 环境的命令），而非替代专用工具。

第十帧：最终确认

Assistant：修复完成。问题原因是注册接口缺少输入验证，特殊字符导致 Django ORM 层抛出异常，被通用异常处理捕获后返回了 500。

修改内容：

1. 添加了 `validate_username()` 函数，限制用户名只能包含字母、数字、下划线和连字符

2. 添加了密码长度验证
3. 区分了验证错误(400)、用户名冲突(409)和服务器内部错误(500)
4. 新增了 4 个测试用例，全部通过

所有 6 个测试通过。

阶段：Feedback（反馈输出）

代理向用户总结了修复内容和验证结果。好的总结应该包含：**做了什么**（事实）、**为什么这么做**（推理）、**怎么验证的**（证据）。

Trace 总结

整个过程涉及了 **10 次工具调用**，可以归纳为以下模式：

初始化 → 理解意图 → 搜索(Glob) → 搜索(Grep) → 阅读(Read×2)
→ 规划 → 编辑(Edit×3) → 测试(Bash) → 总结

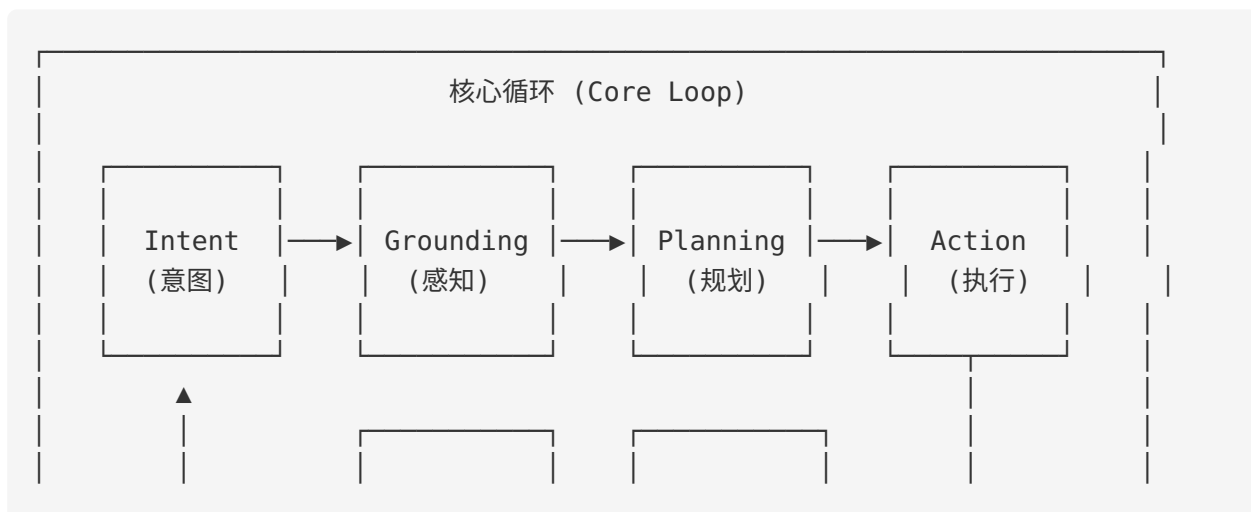
一个关键观察：**代理花在“感知”上的时间（4 次工具调用）与“执行”上的时间（4 次工具调用）大致相当**。这不是巧合——在不确定环境中，充分的感知是高质量行动的前提。

0.2 核心循环

循环的完整推导

上一节的 trace 展示了一个具体实例。现在我们抽象出通用的模式。

AI 编程代理的行为可以分解为一个不断迭代的循环，每次循环包含以下阶段：





各阶段详解

阶段 1: Intent (意图理解)

定义：将用户的自然语言需求转化为代理可操作的内部表征。

这不是简单的“理解用户说了什么”——它涉及以下子任务：

- 需求解析：**从模糊的描述中提取可操作的工程需求
- 范围界定：**判断任务的边界——修改哪些文件、影响哪些功能、需要哪些验证
- 约束识别：**理解隐含的约束条件（如不能破坏现有测试、遵循项目编码规范）
- 目标分解：**将大任务拆分为子目标序列

示例对比：

用户输入	低质量 Intent 理解	高质量 Intent 理解
“这个 API 太慢了”	优化 API	识别慢查询 → 分析执行计划 → 添加索引/缓存 → 验证性能提升
“加个登录功能”	写登录代码	确定认证方案(JWT/Session) → 设计 API → 实现 → 集成前端 → 安全审查
“修复这个 bug”	找到 bug 修复	复现 bug → 定位根因 → 修复 → 回归测试 → 验证修复

工程洞察：Intent 理解的质量很大程度上取决于上下文的丰富程度。这就是为什么 CLAUDE.md、对话历史、项目结构等“元信息”如此重要——它们构成了 Intent 理解的先验知识。

阶段 2: Grounding (感知 — 获取当前状态)

定义：通过工具调用获取代码库的当前状态信息，建立对问题的具体理解。

Grounding 是代理与真实世界的接口。LLM 本身只有训练数据中的“记忆”，但具体项目的代码、结构、状态是实时变化的。Grounding 阶段弥补了“通用知识”与“具体上下文”之间的鸿沟。

常用 Grounding 工具：

工具	用途	信息密度	成本
Glob	文件名模式匹配	低（只有路径）	极低
Grep	内容搜索	中（匹配行）	低
Read	读取文件全文	高（完整内容）	中（占用 context window）
Bash(ls)	目录结构	低	极低
Bash(git log)	变更历史	中	低
Bash(git diff)	差异对比	高	中

Grounding 策略的关键权衡：

- **广度 vs. 深度**：先 Glob 扫描大范围，还是直接 Read 特定文件？
- **信息量 vs. Context 消耗**：读太多文件会耗尽 context window，读太少则可能遗漏关键信息
- **探索 vs. 利用 (Explore vs. Exploit)**：在已知区域深挖，还是搜索新的相关文件？

这个权衡在后续模块中会反复出现——它本质上是一个信息获取的 exploration-exploitation 问题。

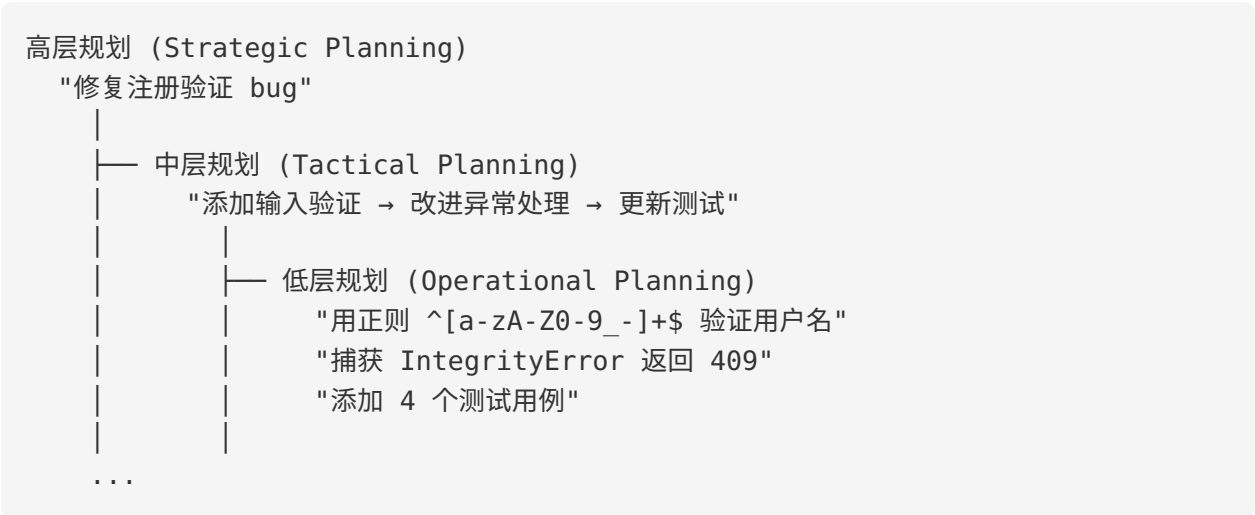
阶段 3：Planning（规划）

定义：基于当前感知到的状态，制定行动方案。

规划可以是显式的（如 Chain-of-Thought 推理）或隐式的（LLM 内部的“直觉”）。好的规划应该包含：

1. **行动序列**：先做什么、后做什么
2. **依赖关系**：哪些步骤之间有依赖
3. **风险评估**：哪些操作可能失败、失败后怎么办
4. **验证标准**：怎么判断方案是否成功

规划的层次：



当前的 AI 编程代理主要依靠 LLM 的单次推理来完成规划（one-shot planning），这既是优势（速度快、端到端）也是局限（缺乏系统性搜索）。后续模块将探讨如何通过搜索算法增强规划能力。

阶段 4：Action（执行）

定义：调用工具对代码库进行修改或执行命令。

Action 是循环中唯一“改变世界”的阶段。其他阶段都是在代理的“思维空间”中运作，只有 Action 会产生外部可观察的效果。

Action 的类型谱系：

类型	示例工具	可逆性	风险等级
只读查询	Glob , Grep , Read	完全可逆	零
文件编辑	Edit , Write	可通过 git 回退	低
命令执行	Bash	取决于命令	低-高
外部交互	git push , API 调用	可能不可逆	高

工程约束： Claude Code 的权限模型对不同类型的 Action 施加了不同的控制——只读操作通常自动批准，而破坏性操作需要用户确认。这不是简单的安全措施，而是一种“行动空间约束”（action space constraint），它通过减少危险动作的可达性来降低出错概率。

阶段 5: Grounding (结果观察)

定义：执行 Action 后，感知执行结果。

这与阶段 2 的 Grounding 使用相同的工具，但目的不同：

- **阶段 2 的 Grounding：**理解问题（“代码现在是什么样的？”）
- **阶段 5 的 Grounding：**验证结果（“我的修改产生了预期效果吗？”）

关键实践：运行测试是最常见的结果观察手段。测试提供了一个明确的、可量化的“信号”：通过/失败。这个信号的价值在于它的可靠性——不像 LLM 的自我评估可能产生幻觉，测试结果是确定性的。

阶段 6: Feedback (反馈)

定义：基于观察到的结果，决定下一步行动——继续循环还是终止。

反馈阶段是循环的“分叉点”：



反馈的质量直接影响收敛速度。一个好的反馈信号应该是： - **具体的：**不只是“失败了”，而是“第 3 个测试在断言 `status_code==400` 时失败，实际值是 500” - **可归因的：**能够追溯到具体的代码变更 - **可操作的：**指出修复方向

循环的宏观特征

1. **迭代性 (Iterative)：**一次循环通常不够——bug 修复可能需要 2-3 轮“编辑-测试-修正”循环
 2. **非均匀性 (Non-uniform)：**不是每个阶段都耗时相同。简单任务可能只需一轮 Grounding + Action；复杂任务可能需要多轮 Grounding
 3. **嵌套性 (Nested)：**大循环中可以包含小循环。例如，在 Planning 阶段可能触发一个“子 Grounding”来获取额外信息
 4. **有界性 (Bounded)：**实际系统中，循环次数受 context window、API 调用次数、用户耐心等资源约束
-

0.3 MDP 建模

为什么要形式化？

将 AI 编程代理的行为形式化为数学模型，不是学术游戏，而是有实际工程价值的：

1. **提供分析框架**：让我们能精确讨论“好策略”和“坏策略”的区别
2. **连接已有理论**：复用强化学习、搜索算法、控制论中数十年的研究成果
3. **指导系统设计**：明确哪些组件可以优化、优化空间有多大
4. **暴露局限性**：理解模型的假设在哪里不成立，从而知道理论的适用边界

MDP 四元组定义

我们将 AI 编程代理建模为 **马尔可夫决策过程 (Markov Decision Process, MDP)**，定义为四元组 (S, A, T, R) ：

S — 状态空间 (State Space)

$$S = (C, H, O, E)$$

其中：

分量	含义	具体内容
C	代码库快照	所有文件的当前内容、目录结构、git 状态
H	对话历史	用户消息、代理回复、系统提示词
O	工具输出缓冲	最近一次工具调用的返回结果
E	环境元数据	工作目录、可用工具列表、权限状态、context window 剩余容量

状态空间的规模：考虑一个中等规模的代码库（10,000 个文件，平均每个文件 200 行，每行 40 个 token），仅代码库部分的状态空间就有约 $V^{(10000 \times 200 \times 40)}$ 种可能（ V 是词汇表大小）。这是一个天文数字，远超围棋的 $3^{361} \approx 10^{172}$ 。

实际处理：LLM 不直接操作完整状态——它只能看到 context window 中的信息。这意味着代理实际操作的是一个被“投影”到有限维度的状态表征。

A — 动作空间 (Action Space)

```
A = {Read(path), Edit(path, old, new), Write(path, content),
      Glob(pattern), Grep(pattern, path),
      Bash(command),
      Message(text),
      ...}
```

动作空间是所有可用工具调用的集合。每个工具调用是一个带有参数的结构化命令。

动作空间的特征：

- 1. **离散但无限**：工具的参数空间是开放的（任意字符串），所以理论上动作空间是无限的
- 2. **结构化**：不像 Atari 游戏的 18 个按钮，编程代理的动作具有丰富的内部结构
- 3. **组合性**：动作可以组合——先 Read 再 Edit 构成一个“读-改”复合动作
- 4. **条件性**：某些动作只在特定状态下可用（如文件不存在时不能 Read）

与围棋的对比：

维度	围棋	AI 编程代理
动作空间大小	~361（每步）	无限（开放参数）
动作结构	扁平（棋盘坐标）	层次化（工具 + 参数）
合法性检查	简单（空位检查）	复杂（语法、语义约束）
每步分支因子	~250	难以估计，极大

T — 状态转移函数 (Transition Function)

```
T: S × A → Δ(S)    // Δ(S) 表示 S 上的概率分布
```

状态转移包含两个性质截然不同的部分：

确定性部分 (Deterministic) — 工具执行：

给定当前代码库状态 C 和一个 Edit 操作，新的代码库状态 C’ 是完全确定的。工具执行是纯函数——相同的输入总是产生相同的输出。


```
Edit("file.py", "old_code", "new_code")
→ 确定性地产生新的文件内容
```

随机性部分（Stochastic）— LLM 生成：

给定当前状态（context window 中的信息），LLM 的下一个动作是从一个概率分布中采样的。这个分布由模型参数、温度等超参数决定。

```
LLM(context)
→ 以某个概率分布生成下一个工具调用
→ temperature=0 时趋近确定性，但并不完全确定
```

混合转移的含义：代理的轨迹（trajectory） $\tau = (s_0, a_0, s_1, a_1, \dots)$ 具有以下特征： - 相同的初始状态和用户指令可能产生不同的执行轨迹 - 轨迹的随机性来自 LLM 的采样过程，而非环境本身 - 这与传统 RL 环境（如 Atari，环境本身是确定性的）有本质区别

R — 奖励函数（Reward Function）

$$R: S \times A \rightarrow \mathbb{R}$$

编程代理的“奖励”没有围棋那样明确的胜负判定。我们需要设计多层次的奖励信号：

即时奖励（Immediate Reward）：

信号	奖励值	可靠性
测试通过	+1.0	高（确定性）
Lint 通过	+0.3	高（确定性）
类型检查通过	+0.3	高（确定性）
编译成功	+0.5	高（确定性）
工具调用成功	+0.1	中
工具调用失败	-0.2	高

延迟奖励（Delayed Reward）：

信号	奖励值	可靠性
用户接受修改	+2.0	高（但延迟大）

信号	奖励值	可靠性
Code Review 通过	+1.5	高（但延迟更大）
部署后无事故	+3.0	高（但延迟极大）

奖励设计的困难：

- 1. **稀疏性 (Sparsity)**：可能执行了 20 步才得到第一个测试结果
- 2. **延迟性 (Delay)**：用户审查可能在几分钟甚至几小时后
- 3. **多目标性 (Multi-objective)**：代码要正确、要可读、要高效、要安全——这些目标之间可能冲突
- 4. **不可度量性 (Immeasurability)**：代码“优雅性”、“可维护性”等质量维度难以自动化量化

部分可观测性：从 MDP 到 POMDP

严格来说，编程代理面对的不是标准 MDP，而是 **部分可观测马尔可夫决策过程 (POMDP)**。

为什么是部分可观测的？

- 1. **代码库不完全可见**：代理不能同时看到所有文件——context window 的限制迫使它每次只能观察一个子集
- 2. **运行时状态不可见**：代码的运行时行为（如数据库内容、网络状态）通常不直接可观察
- 3. **用户意图不完全明确**：用户的需求可能有歧义、有未说出的前提假设
- 4. **依赖关系不透明**：修改一个文件可能影响其他未被阅读的文件

POMDP 扩展：

在 POMDP 框架下，代理维护一个 **信念状态 (belief state)** $b(s)$ ，表示它对真实状态的概率估计：

标准 MDP：代理知道精确状态 s

POMDP：代理维护信念 $b(s) = P(\text{true state} = s \mid \text{observations so far})$

实际影响：

观察到：test_register_success PASSED
test_register_special_chars PASSED
(其他 200 个测试未运行)

信念状态：

$P(\text{全部测试通过}) = ???$

$P(\text{有未发现的回归}) = ???$

代理必须在“运行更多测试以提高信念精度”和“接受当前不确定性以节省时间”之间做权衡。这就是 POMDP 中经典的 **信息获取价值 (Value of Information, Vol)** 问题。

形式化的局限性

任何模型都是现实的简化。MDP/POMDP 框架在以下方面存在局限：

1. **马尔可夫假设**：当前状态并不总是包含所有相关信息。例如，代理在第 5 步看到的代码可能因为 context window 压缩而丢失部分早期信息。严格来说，状态应包含 LLM 的完整 context window，但这本身就是一个近似。
2. **奖励函数的主观性**：上面列出的奖励值是设计选择，不是客观事实。不同的奖励设计会导致不同的最优策略。
3. **状态空间的连续性**：代码变更不是真正“离散”的——一个变量名从 `x` 改为 `count` 在语义上是连续的改进，但在形式化中被当作离散状态跳转。
4. **多代理交互**：在团队协作场景中（如多个代理并行工作），MDP 需要扩展为博弈论框架（Markov Game）。

尽管如此，MDP 框架仍然是目前最好的分析工具——它提供了足够的结构来指导系统设计，同时保持了数学上的可操作性。我们只需记住它的边界。

0.4 策略空间总览

“地图”的意义

在前面的讨论中，我们定义了核心循环的各个阶段和 MDP 的基本要素。现在，我们需要一张“地图”——一个全景视图，展示在每个阶段、代理可以选择哪些策略，以及每种策略的代表性工具/论文和核心权衡。

这张表是整个课程的索引：后续每个模块都会深入某一行或某一列。

策略全景表

环节	策略选项	代表工具/论文	核心权衡
Intent（意图理解）			
	直接执行	基础 prompt engineering	速度快 vs. 可能误解复杂需求
	澄清对话	Claude Code AskUserQuestion	准确性高 vs. 打断用户流
	上下文增强	CLAUDE.md, RAG	信息丰富 vs. context 消耗
	任务分解	Plan mode, TodoWrite	结构清晰 vs. 分解开销
Grounding（感知）			
	粗粒度搜索	Glob , ls	速度快 vs. 可能遗漏
	细粒度搜索	Grep , Read	精确 vs. 成本高
	结构化索引	AST 解析, Tree-sitter, LSP	语义准确 vs. 构建成本
	向量检索	Embedding + RAG	语义匹配 vs. 索引维护
	代理化搜索	Agentic Explore (子代理)	深度探索 vs. 延迟和开销
Planning（规划）			
	单步规划（贪心）	标准 LLM 推理	速度快 vs. 局部最优
	Chain-of-Thought	Wei et al., 2022	推理质量高 vs. token 消耗
	多步规划	Plan mode, 结构化规划	全局视野 vs. 规划成本
	树搜索	MCTS, ToT (Yao 2023)	探索充分 vs. 计算昂贵
	分层规划	HTN, 任务分解	可扩展 vs. 分解困难

环节	策略选项	代表工具/论文	核心权衡
Action (执行)			
	单次编辑	Edit (精确替换)	安全 vs. 可能需多次调用
	文件重写	Write (全文覆盖)	快速 vs. 风险高
	批量操作	多工具并行调用	效率高 vs. 依赖管理复杂
	子代理委派	Agent (团队模式)	并行化 vs. 协调开销
	交互式执行	Bash (运行脚本/测试)	灵活 vs. 安全风险
Grounding (结果观察)			
	测试驱动	pytest, jest, CI	可靠 vs. 需要测试存在
	静态分析	Lint, Type check	快速 vs. 覆盖有限
	LLM 自评估	自我反思 (Reflexion)	灵活 vs. 可能幻觉
	人工审查	用户确认	最可靠 vs. 最慢
Feedback (反馈)			
	立即重试	简单循环	实现简单 vs. 可能无限循环
	错误分析后重试	Reflexion (Shinn 2023)	质量高 vs. 额外推理成本
	回滚重规划	Git reset + 重新规划	避免沉没成本 vs. 效率低
	升级求助	交回用户控制	安全 vs. 用户负担

如何使用这张表

- 诊断当前系统：**对照表格，分析你的代理在每个环节使用了哪种策略。当前大多数系统在所有环节都使用最简单的策略（第一行）——这意味着有大量优化空间。

- 2. **识别瓶颈**：如果代理在某个任务上表现不佳，对照表格定位是哪个环节出了问题。是 Grounding 不充分（没找到相关代码）？还是 Planning 质量不够（方案有缺陷）？还是 Feedback 信号太弱（没有好的验证方式）？
- 3. **设计改进方案**：针对瓶颈环节，选择更高级的策略。但注意每种策略都有成本——不是越高级越好，而是要匹配任务复杂度。
- 4. **理解课程结构**：后续模块将依次深入表格中的各个策略，提供理论分析和工程实现。

复杂度梯度

一个有用的思维模型是“策略复杂度梯度”：



关键决策点是：**何时从简单策略“升级”到复杂策略？** 这本身就是一个元策略（meta-strategy）问题，我们将在后续模块中深入讨论。

0.5 与 AlphaGo 的结构类比

为什么要做这个类比？

AlphaGo 是公众认知中“AI 解决复杂决策问题”的标志性案例。通过与之类比，我们可以：

- 1. 利用已有的直觉来理解编程代理的结构
- 2. 识别编程代理中“缺失的组件”——即理论上应该有但当前系统尚未实现的部分
- 3. 建立一个讨论改进方向的语言

组件对应关系

Policy Network $\approx$ LLM（策略网络 $\approx$ 大语言模型）

AlphaGo 的策略网络： - 输入：当前棋盘状态（ 19×19 的多通道特征图） - 输出：下一步落子的概率分布（361 个位置的概率） - 作用：生成“直觉上好的”候选走法

编程代理的 LLM： - 输入：当前 context window（系统提示 + 对话历史 + 工具输出） - 输出：下一个动作的概率分布（工具调用 + 参数） - 作用：生成“直觉上好的”下一步操作

类比成立的地方： - 两者都是从大量数据中学习的函数近似器 - 两者都将复杂状态映射为动作概率分布 - 两者都在推理时（inference time）实时做出决策 - 两者的“直觉”质量都取决于训练数据的质量和数量

类比不成立的地方： - LLM 的动作空间远大于围棋（开放文本 vs. 361 个固定位置） - LLM 有“推理能力”（reasoning capabilities）——它可以在生成动作之前进行显式的逻辑推理，而策略网络只做模式匹配 - LLM 的输出空间包含“思考”和“行动”两部分（参见 ReAct），策略网络只输出行动

Value Network $\approx$ 测试/评估（价值网络 $\approx$ 验证信号）

AlphaGo 的价值网络： - 输入：当前棋盘状态 - 输出：当前局面的胜率估计（0 到 1 之间的标量） - 作用：评估“当前局面有多好”，无需走到终局

编程代理的测试/评估： - 输入：当前代码库状态 - 输出：各种质量信号（测试通过率、lint 结果、类型检查结果） - 作用：评估“当前代码有多好”，无需部署到生产环境

类比成立的地方： - 两者都提供了对“当前状态质量”的估计 - 两者都帮助在搜索过程中进行“剪枝”——排除明显差的方向 - 两者都是替代“完整评估”（走到终局/部署到生产）的近似方法

类比不成立的地方：

维度	AlphaGo 价值网络	编程代理测试
获取成本	极低（前向传播一次）	中-高（需要运行测试套件）
全面性	单一标量（胜率）	多维信号（各种测试结果）
可微性	是（可用于训练）	否（离散的通过/失败）

维度	AlphaGo 价值网络	编程代理测试
存在性	总是存在	不一定有（项目可能缺少测试）
准确性	训练后很高	取决于测试质量

关键洞察：编程领域缺乏一个通用的、低成本的价值评估器。这是当前系统的核心瓶颈之一。围棋的价值网络可以在毫秒内评估任何局面，但编程中，运行测试可能需要几秒到几分钟——这使得大规模搜索变得不可行。

MCTS ≈ 搜索策略（蒙特卡洛树搜索 ≈ 探索策略）

AlphaGo 的 MCTS： - 从当前局面出发，探索未来可能的走法序列 - 用策略网络指导探索方向（哪些走法值得尝试） - 用价值网络和快速走子评估每个探索分支的前景 - 综合多次模拟的结果，选择最佳走法

编程代理的“搜索策略”：

这是当前系统中**最薄弱的环节**。

当前主流的 AI 编程代理（Claude Code、Cursor、GitHub Copilot Agent）基本上是**贪心的**：

```
# 当前系统的伪代码
while not done:
    action = LLM.generate(context)      # 策略网络：直接采样一个动作
    result = execute(action)            # 执行
    context = update(context, result)    # 更新状态
    # 没有搜索！没有回溯！没有多路径探索！
```

这相当于用 AlphaGo 的策略网络直接下棋，**完全不做搜索**。

为什么当前系统不做搜索？

- 1. **Action 成本高：**围棋的模拟走子是瞬时的，但编程中的工具调用（尤其是测试）需要真实时间
- 2. **状态回滚困难：**围棋可以轻松“悔棋”回到之前的局面，但代码修改可能涉及文件系统、数据库等有副作用的操作
- 3. **分支因子过大：**每一步的候选动作太多，全面搜索不可行
- 4. **缺乏价值网络：**没有快速评估中间状态的方法

改进方向（预告后续模块）：

尽管存在上述困难，一些先驱性的工作已经开始探索搜索策略：

- **SWE-Search（后续模块详讲）**：将 MCTS 适配到软件工程任务，使用 LLM 作为价值函数
- **Tree of Thoughts（Yao 2023）**：在推理过程中构建思维树，探索多个推理路径
- **CodeR（后续模块详讲）**：多代理协作，通过角色分工实现隐式搜索
- **Git-based rollback**：利用版本控制系统实现低成本的状态回滚

核心洞察：贪心的代价

让我们用一个具体例子说明“贪心”策略的代价：

任务：修复一个涉及三个模块交互的 bug

贪心策略的轨迹：

- Step 1: 修改模块 A ✓（看起来修复了）
- Step 2: 运行测试 → 模块 B 的测试失败了
- Step 3: 修改模块 B 来适配 A 的改动 ✓
- Step 4: 运行测试 → 模块 C 的测试失败了
- Step 5: 修改模块 C... 但现在改动太大了，引入了新 bug
- Step 6: 反复折腾...
- 总计：15 步，最终勉强修复

搜索策略的轨迹（理想情况）：

- Step 1: 分析三个模块的交互关系
- Step 2: 评估三种修复方案（改 A、改 B、或改 A-B 接口）
- Step 3: 选择影响最小的方案（改接口）
- Step 4: 实施修改
- Step 5: 运行测试 → 全部通过
- 总计：5 步，干净利落

数学表达：设贪心策略的期望步数为 $E[\text{steps_greedy}]$ ，搜索策略为 $E[\text{steps_search}]$ 。在任务复杂度增长时，两者的差距呈超线性增长：

$$\begin{aligned} E[\text{steps_greedy}] &= O(\text{complexity} \times \text{branching_factor}) \\ E[\text{steps_search}] &= O(\text{complexity} \times \log(\text{branching_factor})) \end{aligned}$$

这就是搜索的价值——用前期的“思考时间”换取后期的“执行效率”。

对比总结表

组件	AlphaGo	AI 编程代理	差距
策略网络	训练专用的 CNN	通用 LLM	代理的策略网络更通用，但不如专用网络精确
价值网络	训练专用的 CNN	测试套件 + 静态分析	代理缺乏快速、通用的状态评估
搜索	MCTS (数千次模拟)	几乎没有 (贪心)	代理的最大短板
训练	自我对弈 + RL	预训练 + RLHF	代理缺乏任务特定的强化学习
环境	确定性、完全信息	随机性、部分可观测	代理面对更困难的环境
回合长度	~200 步	5-50 步 (通常)	代理的每步成本高得多

核心结论：当前的 AI 编程代理大致相当于“只有策略网络、没有搜索”的 AlphaGo——这已经比人类业余棋手强（能处理大多数常规编程任务），但要达到“超人”水平（可靠地处理复杂工程任务），搜索和评估机制的引入是必要的。

关键论文导读

ReAct: Synergizing Reasoning and Acting in Language Models

论文信息： Yao et al., 2022. “ReAct: Synergizing Reasoning and Acting in Language Models.” ICLR 2023.

核心思想

ReAct 的核心观察是：**推理（Reasoning）和行动（Acting）应该交错进行，而不是分离的。**

在 ReAct 之前，存在两种范式：

范式 1：纯推理 (Reasoning-only)

思考 → 思考 → 思考 → ... → 最终答案

问题：没有外部信息验证，容易产生幻觉

范式 2：纯行动 (Acting-only)

行动 → 观察 → 行动 → 观察 → ...

问题：没有高层推理指导，行动可能盲目

ReAct 将两者统一：

范式 3：ReAct (推理 + 行动交错)

思考 → 行动 → 观察 → 思考 → 行动 → 观察 → ...

优势：推理指导行动，观察校正推理

具体格式

ReAct 为 LLM 的输出定义了三种类型的“步骤”：

```
Thought: 我需要找到注册接口的代码，先搜索包含 "register" 的文件
Action: Search["register function python"]
Observation: Found in app/views/register.py, line 45: def
register_user(request)
Thought: 找到了注册函数，需要阅读它来理解当前的验证逻辑
Action: Read["app/views/register.py"]
Observation: [文件内容]
Thought: 发现没有输入验证，特殊字符直接传给 create_user 导致异常
Action: Edit["app/views/register.py", ...]
```

注意每个 Thought 都起到了承上启下的作用：- **向后**：解释前一个 Observation 的含义（“发现没有输入验证”）- **向前**：指导下一个 Action 的选择（“需要阅读它来理解”）

与 Claude Code 的关系

Claude Code 的核心循环本质上是 ReAct 范式的工程实现：

```
Claude Code 的 "thinking" 块 ≈ ReAct 的 Thought
Claude Code 的工具调用       ≈ ReAct 的 Action
Claude Code 的工具返回结果    ≈ ReAct 的 Observation
```

但 Claude Code 在 ReAct 基础上做了大量工程增强：- **多工具并行调用**：ReAct 是严格串行的，Claude Code 允许无依赖的工具调用并行执行 - **结构化工具接口**：ReAct 的 Action 是自由文本，Claude Code 的工具调用是带参数类型的结构化 API - **权限控制**：

ReAct 没有权限概念，Claude Code 对不同风险等级的操作有不同的审批流程 - **Context 管理**：ReAct 的 context 会线性增长，Claude Code 有压缩和裁剪机制

学术贡献

ReAct 的价值不在于算法的复杂性，而在于：

1. **概念统一**：将推理和行动视为同一框架内的两种互补操作
2. **实验验证**：在多种任务上证明了交错策略优于纯推理或纯行动
3. **可解释性**：Thought 步骤提供了可审查的推理链，增加了系统的透明度
4. **范式确立**：几乎所有后续的 LLM agent 工作都建立在 ReAct 范式之上

Chain-of-Thought Prompting Elicits Reasoning in Large Language Models

论文信息：Wei et al., 2022. “Chain-of-Thought Prompting Elicits Reasoning in Large Language Models.” NeurIPS 2022.

核心思想

Chain-of-Thought (CoT) 的核心发现是：**通过在提示中展示“思考过程”的示例，可以显著提升 LLM 在推理任务上的表现。**

标准提示 vs. CoT 提示对比：

标准提示：

Q：一个书架有 3 层，每层放 8 本书。拿走 5 本后还有多少？
A：19

CoT 提示：

Q：一个书架有 3 层，每层放 8 本书。拿走 5 本后还有多少？
A：书架共有 3 层，每层 8 本，所以总共有 $3 \times 8 = 24$ 本。
拿走 5 本后，还剩 $24 - 5 = 19$ 本。
答案是 19。

为什么 CoT 对编程代理重要？

编程代理的每一步决策都涉及复杂推理：

不使用 CoT 的代理：

User: 这个接口太慢了
Agent: [直接开始修改代码, 可能改错方向]

使用 CoT 的代理:

User: 这个接口太慢了
Agent (thinking):
接口慢可能有几个原因:
1. 数据库查询效率低 (N+1 查询、缺少索引)
2. 序列化逻辑复杂 (嵌套关系序列化)
3. 外部 API 调用阻塞

首先需要确认瓶颈在哪里。查看接口代码和数据库查询是第一步。
Agent: [有针对性地开始诊断]

CoT 在编程代理中的价值:

- 分步推理降低错误率:** 复杂的代码修改需要多步推理, 逐步分解降低了跳步出错的概率
- 显式化前提假设:** CoT 迫使代理写出它的假设 (如 “假设这是一个 Django 项目”), 这些假设可以被验证或纠正
- 可审查性:** 用户可以阅读代理的思考过程, 发现推理错误并及时纠正
- 自我校正:** 在写出推理链的过程中, LLM 有机会 “发现” 自己之前的错误 (类似人类 “说出来才发现哪里不对”)

CoT 的变体与演进

基础 CoT (Wei 2022)

- Zero-shot CoT ("Let's think step by step")
适用场景: 简单推理任务
- Self-Consistency (Wang 2022)
思想: 采样多条推理链, 多数投票选答案
适用场景: 有明确答案的问题
- Tree of Thoughts (Yao 2023)
思想: 将 CoT 从链扩展为树, 允许回溯
适用场景: 需要探索多条路径的规划问题
- ReAct (Yao 2022)

思想：CoT + 外部工具交互
适用场景：需要外部信息的推理任务（如编程）

CoT 是 ReAct 的理论基础——ReAct 可以看作“带外部行动能力的 CoT”。

从理论到实践

在当前的 AI 编程代理中，CoT 以多种形式存在：

1. **显式 thinking 块**：Claude Code 在每次工具调用前会生成思考过程（`thinking field`），这是最直接的 CoT 应用
2. **系统提示中的推理指令**：如“在修改代码之前，先分析问题的根本原因”
3. **Plan mode**：要求代理先产出完整的修改计划再执行，是 CoT 在更粗粒度上的应用
4. **Commit message / PR description**：描述“为什么这么改”而非“改了什么”，是 CoT 思想在工程实践中的体现

实操环节

练习：标注 Claude Code Trace

练习目标

通过亲手标注一个真实的 Claude Code 执行 trace，将理论概念（核心循环的各个阶段）与实际行为建立直接映射。

准备工作

1. 确保已安装 Claude Code CLI
2. 准备一个包含 bug 的小型项目（或使用下面提供的练习项目）

练习项目设置

创建以下文件结构：

```
mkdir -p exercise/src exercise/tests
```

`exercise/src/calculator.py`：

```

class Calculator:
    def __init__(self):
        self.history = []

    def add(self, a, b):
        result = a + b
        self.history.append(f"{a} + {b} = {result}")
        return result

    def divide(self, a, b):
        result = a / b # Bug: no zero division handling
        self.history.append(f"{a} / {b} = {result}")
        return result

    def get_history(self):
        return self.history

    def average(self, numbers):
        total = 0
        for n in numbers:
            total += n
        return total / len(numbers) # Bug: empty list not handled

    def clear_history(self):
        self.history = None # Bug: should be empty list, not None

```

exercise/tests/test_calculator.py :

```

import pytest
from src.calculator import Calculator

class TestCalculator:
    def setup_method(self):
        self.calc = Calculator()

    def test_add(self):
        assert self.calc.add(2, 3) == 5

    def test_divide(self):
        assert self.calc.divide(10, 2) == 5.0

    def test_divide_by_zero(self):
        with pytest.raises(ValueError, match="Cannot divide by zero"):
            self.calc.divide(10, 0) # Will fail: raises
ZeroDivisionError, not ValueError

    def test_average(self):
        assert self.calc.average([1, 2, 3]) == 2.0

```

```
def test_average_empty(self):
    with pytest.raises(ValueError, match="Cannot average empty list"):
        self.calc.average([]) # Will fail: raises ZeroDivisionError

def test_history(self):
    self.calc.add(1, 2)
    self.calc.divide(6, 3)
    assert len(self.calc.get_history()) == 2

def test_clear_history(self):
    self.calc.add(1, 2)
    self.calc.clear_history()
    assert len(self.calc.get_history()) == 0 # Will fail: None has no
len()
```

练习步骤

Step 1: 运行任务并捕获 trace

启动 Claude Code，给出以下指令：

请修复 calculator.py 中的所有 bug，确保所有测试通过。

在执行过程中记录每一步的操作。

Step 2: 创建标注表

使用以下模板记录每一步：

步骤编号	工具调用	输入参数	核心循环阶段	你的分析
1	Glob	**/*calculator*	Grounding（感知）	定位相关文件
2	Read	src/calculator.py	Grounding（深度感知）	阅读源代码
3	Read	tests/test_calculator.py	Grounding（补充上下文）	理解测试期望
4	(thinking)	—	Planning	制定修复方案

步骤编号	工具调用	输入参数	核心循环阶段	你的分析
5	Bash	"pytest tests/ -v"	Grounding（基线测量）	确认当前失败状态
6	Edit	calculator.py: divide	Action	添加除零检查
7	Edit	calculator.py: average	Action	添加空列表检查
8	Edit	calculator.py: clear_history	Action	修复为空列表
9	Bash	"pytest tests/ -v"	Grounding（结果观察）	验证修复
10	(text output)	—	Feedback	向用户报告结果

Step 3：回答分析问题

完成标注后，回答以下问题：

- 1. **Grounding** 步骤占总步骤的比例是多少？ 这个比例是否出乎你的意料？
- 2. 代理是否在执行修复之前先运行了测试（建立基线）？ 如果没有，这会影响什么？
- 3. 代理是否做了多轮循环（修复-测试-修复）？ 如果是，第二轮循环的原因是什么？
- 4. 对照策略空间总览表（0.4 节），代理在每个阶段使用了哪种策略？是最简单的还是更高级的？
- 5. 你能想到代理可以用更高效的策略来完成同一任务吗？ 写出你的替代方案。

Step 4（进阶）：比较不同 prompt 策略

用两种不同的方式给出同一个任务，观察代理行为的差异：

Prompt A（模糊）： "代码有 bug，请修复"

Prompt B（精确）： "calculator.py 中的 divide、average 和 clear_history 方法存在 bug：divide 缺少除零处理，average 缺少空列表处理，clear_history 应设为空列表而非 None。请修复并确保所有测试通过。"

对比两次执行的： - 步骤数 - Grounding 阶段的深度和广度 - 最终结果质量 - 总耗时

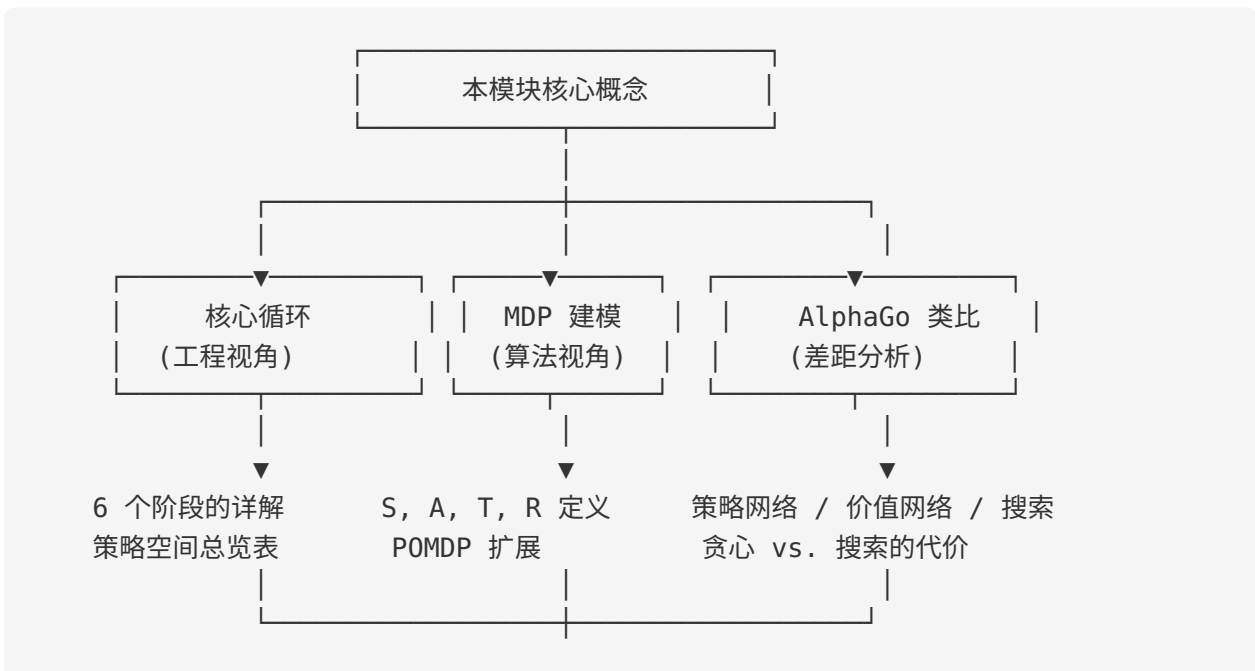
预期发现： Prompt B 会减少 Grounding 步骤（因为用户已经提供了定位信息），但可能产生“隧道视野”——代理可能不会检查是否还有其他 bug。Prompt A 更费时，但可能发现更多问题。这呼应了我们在 Intent 阶段讨论的权衡。

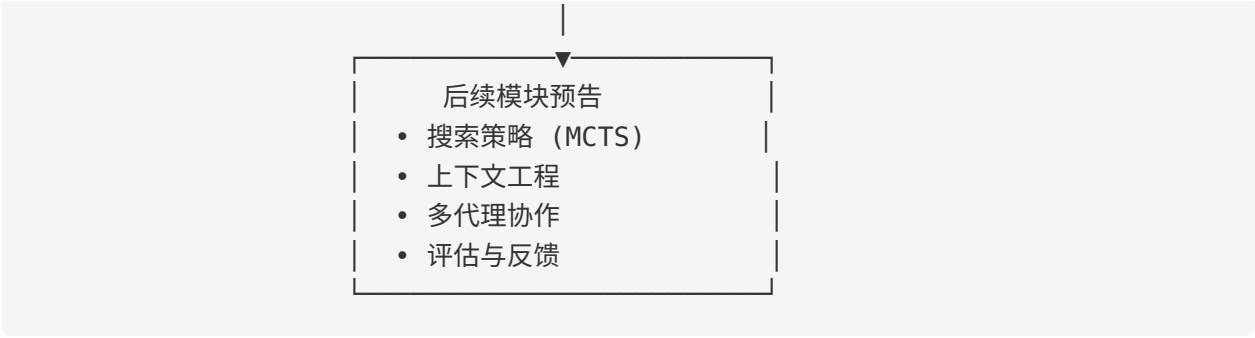
本模块小结

关键点

- 核心循环是基础：** AI 编程代理的所有行为都可以分解为 Intent → Grounding → Planning → Action → Grounding → Feedback 的迭代循环。理解这个循环是理解一切高级策略的前提。
- MDP 提供分析框架：** 将编程代理建模为 (S, A, T, R) 四元组，让我们能用数学语言讨论策略的优劣。但要注意部分可观测性 (POMDP) 和模型假设的局限性。
- 当前系统是贪心的：** 与 AlphaGo 的对比揭示了当前系统的结构性缺陷——有策略网络 (LLM)，有价值信号 (测试)，但几乎没有搜索。这是最大的改进空间。
- 策略空间是广阔的：** 每个阶段都有从简单到复杂的多种策略选项。选择合适的策略组合是系统设计的核心挑战。
- ReAct 和 CoT 是基石：** 推理与行动的交错 (ReAct) 和逐步推理 (CoT) 是当前所有 AI 编程代理的理论基础。理解它们的原理和局限，是改进系统的起点。

概念图





连接到后续模块

本模块概念	后续深入模块
Grounding 策略	上下文工程 (Context Engineering)
Planning 策略	搜索与规划 (Search & Planning)
Action 策略	工具使用与代码生成 (Tool Use & Code Generation)
贪心 vs. 搜索	搜索增强策略 (Search-Augmented Strategies)
多代理	多代理协作 (Multi-Agent Collaboration)
奖励与评估	评估与基准测试 (Evaluation & Benchmarking)

思考题

问题 1：Context Window 的经济学

假设你有一个 200K token 的 context window，需要修复一个涉及 5 个文件的 bug。每个文件平均 500 行（约 2000 token）。

- 你会一次性读取所有 5 个文件（消耗 10K token），还是逐个读取、处理完一个再读下一个？
- 分析两种策略在不同场景下的优劣（考虑文件间依赖关系的强弱）。
- 如何将这个问题形式化为 MDP 中的一个决策？状态是什么？动作是什么？奖励函数如何设计？

问题 2：搜索的成本与价值

在 AlphaGo 中，搜索（MCTS）将棋力提升了数个段位。但在编程代理中，搜索的引入面临独特的挑战。

- 列举至少 3 个使编程领域搜索更困难的因素，并逐一分析可能的解决方案。
- 假设你要为 Claude Code 添加一个简单的“搜索”机制，允许它在两个修复方案之间比较（而非只尝试第一个），你会如何设计？需要什么额外的基础设施？
- 估算这种搜索机制的成本/收益比（提示：考虑 API 调用次数、执行时间、准确率提升）。

问题 3：贪心何时足够好？

我们论证了当前系统是“贪心的”，并暗示这是一个缺陷。但贪心策略在很多实际场景中表现良好。

- 描述 3 种编程任务场景，其中贪心策略（直接执行 LLM 的第一个建议）就足够好了。这些场景有什么共同特征？
- 描述 3 种编程任务场景，其中贪心策略明显不够好，需要某种形式的搜索。这些场景有什么共同特征？
- 能否设计一个“元策略”来自动判断何时应该从贪心切换到搜索？它需要什么信号？

问题 4：奖励函数设计

你正在为一个编程代理设计奖励函数。该代理的任务是进行代码重构——不改变功能，只改善代码质量。

- 列举所有你能想到的可自动化计算的代码质量信号，并评估每个信号的可靠性。
- “所有测试通过”是重构任务的奖励信号吗？它足够吗？它可能遗漏什么？
- 如何处理多个质量信号之间的冲突（例如：更短的代码 vs. 更可读的代码）？
- 讨论使用 LLM 本身作为“奖励模型”（判断代码质量）的可行性和风险。

问题 5（开放性）：POMDP 视角下的工具设计

从 POMDP 的视角看，代理的每一次工具调用都是一次“观测”——它减少了状态的不确定性。

- 如果你要设计一个新的开发者工具（为 AI 编程代理优化，而非为人类优化），它应该具备什么特征？它应该返回什么样的信息才能最大化“信息价值”？

- 当前的 `Grep` 工具返回匹配行。一个“为 AI 优化”的 `Grep` 可能还应该返回什么额外信息？（提示：考虑减少后续工具调用的需要）
- 讨论“为 AI 优化工具”和“为人类优化工具”之间的共同点与差异。长期来看，这两类工具会趋同还是分化？